

BMVEI Library Management System - Technical Documentation

Bishoftu Motor Vehicle Engineering Industry (BMVEI) *Document Type: Technical Documentation | Version: 1.0.0 | Status: Released*

1. Project Overview

1.1 Purpose & Scope

The **BMVEI Library Management System (LMS)** is a full-stack enterprise web application developed to replace the paper-based library record-keeping process at Bishoftu Motor Vehicle Engineering Industry. It provides a centralized, digital platform for managing the physical book catalog, member borrowing, reservations, and reporting.

1.2 System Goals

- **Operational:** Digitize borrowing lifecycle, track physical copies in real-time, automate overdue flagging via nightly jobs, and support bulk catalog imports.
 - **Governance:** Complete immutable audit trail, Role-Based Access Control (RBAC), secure credential management, CSV reporting, and configurable system parameters.
-

2. System Architecture

2.1 Technology Stack

- **Frontend:** React + Vite, Tailwind CSS, Framer Motion, Zustand (State), TanStack Query, React Hook Form + Zod.
- **Backend:** Node.js, Express, MySQL 8.0, mysql2, JWT, bcrypt.
- **Infrastructure:** Nginx (Reverse Proxy), PM2 (Process Manager), node-cron (Jobs).

2.2 Architectural Pattern

The backend follows a **Layered Architecture**:

1. **Middleware:** JWT auth → RBAC → Input validation → Rate limiter.
 2. **Controller:** Reads validated input, calls service, formats response. No business logic.
 3. **Service:** Owns all business rules, quota checks, transaction coordination.
 4. **Repository:** Executes parameterized SQL queries.
-

3. Database Design

The database is normalized to 3NF using MySQL 8.0 with `utf8mb4` encoding. All relationships use foreign key constraints (InnoDB).

3.1 Key Tables

- `users` : All accounts (admins, librarians, employees). Contains `password_hash`, `role_id`, and `department_id`.
 - `books` : Central catalog tracking `total_copies` and `available_copies`.
 - `borrow_records` : Full loan lifecycle from issue to return. Tracks `status` (borrowed/returned/overdue/lost).
 - `reservations` : Hold queue for unavailable books.
 - `audit_logs` : Immutable action audit trail.
-

4. API Reference

All endpoints are prefixed with `/api/v1`. Protected routes require `Authorization: Bearer <token>`.

4.1 Authentication (`/auth`)

- `POST /auth/login` - Authenticate, get access token & HttpOnly refresh cookie.
- `POST /auth/logout` - Clear refresh token.
- `POST /auth/refresh` - Issue new access token.

4.2 Books (`/books`)

- `GET /books` - List with search/filter/pagination.
- `POST /books` - Create book (supports multipart/form-data for cover).
- `POST /books/bulk` - Bulk import via JSON.

4.3 Borrow Records (`/borrow-records`)

- `POST /borrow-records/issue` - Issue a book (validates quotas atomically).
- `POST /borrow-records/:id/return` - Process return.

4.4 Reservations (`/reservations`)

- `POST /reservations` - Reserve unavailable book.
- `POST /reservations/:id/fulfill` - Mark fulfilled (automatically creates borrow record and notifies user).

5. Core Business Logic

5.1 Borrowing Lifecycle (Atomic Transaction)

1. Validate user has fewer than `max_books_per_user` active loans.
2. Validate user does not already have this specific book borrowed.
3. `SELECT available_copies FOR UPDATE` (row-level lock).
4. Verify `available_copies >= 1`.
5. `INSERT borrow_records` and `UPDATE books SET available_copies = available_copies - 1`.
6. Commit transaction and log audit trail.

5.2 Reservation Fulfillment

- Reservations can only be made if `available_copies == 0`.
- When a librarian fulfills a reservation, the system automatically calls `borrowService.issueBook()` internally.
- The reservation status changes to fulfilled, a borrow record is created, and the user's notification feed is updated.

5.3 Overdue Management

A `node-cron` job runs nightly, atomically updating all borrowed records past their due date to `overdue`.

6. Security Architecture

- **JWT Dual-Token Pattern:** Short-lived access token in memory, refresh token in `HttpOnly` cookie to prevent XSS theft.

- **Password Security:** Bcrypt hashing (cost factor 12) and strict enforcement via `Zod/express-validator`.
 - **Brute-force Protection:** Account lockout after 5 failed login attempts.
 - **SQL Injection Prevention:** 100% parameterized queries via `mysql2`.
 - **XSS & HTTP Headers:** Input sanitization and `Helmet` for security headers.
-

7. Deployment Guide

7.1 Local Development

```
# Database Setup
mysql -u root -e "CREATE DATABASE bmvei_lms;"

# Backend
cd backend
npm install
npm run migrate
npm run seed
npm run dev

# Frontend
cd frontend
npm install
npm run dev
```

7.2 Production (LAN Server)

1. Build frontend: `cd frontend && npm run build`
 2. Start backend via PM2: `pm2 start src/server.js --name bmvei-lms-api`
 3. Configure Nginx to serve static files from `/frontend/dist` and proxy `/api/` traffic to `http://localhost:5000`.
-

© 2026 Bishoftu Motor Vehicle Engineering Industry · Internal Use Only